

Operating Systems

Ingyu Bahng

June 2026

Contents

1	Windows	2
2	macOS	2
2.1	Foundational Software	2
2.2	macOS Storage Architecture	2
3	Linux	2
3.1	Foundational Software	2
3.2	General Distribution Components	3
3.3	Linux File System Hierarchy	6
3.4	Partitioning & Disk Management	10
3.5	The Boot Process	10
3.6	Display Server & GPU Drivers	10
3.7	Networking Setup	10
3.8	Package Management	10
3.9	Major Linux Distributions	10
3.9.1	Ubuntu	10
3.9.2	Mint	10
3.9.3	Arch	10

1 Windows

2 macOS

2.1 Foundational Software

2.2 macOS Storage Architecture

Core Concept 2.1: Root Directory & Overview

Core Concept 2.2: Disk Storage Management

need to clear out caches (16GB) and look into app support (21 GB) more carefully

/Applications/ 8.85GB /System/ 36GB - BULK≥ /Library/ 16.14 GB - here you can clean out Logs/ and Caches/
/opt/ - can cleanup cached downloads brew keeps after installing = "brew cleanup" /private/var/log/* - can
clean this up /Users/ 74.43

3 Linux

3.1 Foundational Software

Core Concept 3.1: Linux Kernel

Con conversationally, people refer to linux as the linux operating system, however linux actually refers to the **linux kernel**, a fundamental piece of software that acts as a middle man between the computer's hardware (CPU, RAM, disk) and the software (Chrome, Neovim, or Python scripts). It broadly manages four key areas in a machine.

- **Process Management:** It decides which program gets to use the CPU and for how long.
- **Memory Management:** It keeps track of which RAM portion is being used by which app and ensures no overlap.
- **Device Management:** It communicates with hardware (keyboard, mouse, etc.) and its signals via **drivers** (Core Concept 3.3).
- **System Calls:** It handles requests from software called "system calls", such as python code wanting to save a file to the disk.

At face value, the linux kernel is machine code (binary) that has been compiled from C and Assembly written by **Linus Torvalds** in 1991.

Core Concept 3.2: GNU Operating System

GNU is actual operating system of the "linux operating system", which is a collection of the following software.

- **The compiler (GCC)** is the translator of C code into machine code.
- **The shell** is the command line interface (CLI) used to talk directly to the kernel via commands defined in the core utilities primarily residing in `/bin` and `/sbin` (Core Concept 3.12).
- **The C library (glibc)** is the fundamental set of instructions written in C and some Assembly—hence the

direct need of the compiler GCC to arrive with the operating system—that almost every other base program on the computer relies onto do basic things like opening a file.

- **Core utilities** refer to the basic CLI commands such as `ls`, `rm`, and `grep`, which, at face value, are no extension files written purely in machine code.

Core Concept 3.3: Drivers

Modern operating systems have to communicate with thousands of different hardware components (*mouse, keyboard, wifi chip, printer, graphics card, etc.*). This is where **drivers** come in.

A **driver** is a specialized piece of software that tells the kernel how to communicate with a specific piece of hardware, sitting **between the kernel and the hardware**.

User Apps → glibc → System Call → Linux Kernel [Driver] → Hardware

There are two different types of drivers in the context of linux.

- **Monolithic (built in) drivers** are part of the linux kernel file, making it very fast (*keyboard, disk, etc.*).
- **Kernel module drivers** are only loaded in when needed, keeping the kernel small and efficient (*like a USB webcam*). These module drivers are kept in `/lib/modules`.

It's important to know that **every unique piece of hardware requires a complementary driver** in order to be accessed by the linux kernel. Thus, if the complementary driver to any cutting edge piece of hardware doesn't exist in the local, one would have to manually install the driver before being able to use it.

The linux kernel is able to search whether it has the specific driver for a piece of hardware via a tiny unchangeable chip called the **ID chip**, which simply says something like *"I am a Logitech Webcam model 9000"*. If a hardware is connected to the machine, the kernel subsequently reads the ID chip and sees if it had its corresponding driver within its kernel modules.

3.2 General Distribution Components

A linux distribution is essentially the linux kernel + GNU OS + other software that give each distro their unique personality.

The other software are the collection of the following categories

Core Concept 3.4: Init System

When a kernel finishes loading, it doesn't necessarily know how to start your machine (desktop, wifi, etc.). It instead starts one program (PID 1) called the init system which starts all of the background processes needed to just turn on the computer for the user. At face value, systemd is a suite of no extension files written in C and compiled with GCC.

Most modern and major distros use an init system called systemd. In certain niche cases, other init systems are used like openrc, runit, but these are irrelevant for just the average linux user.

The linux kernel knows to run systemd first because it is hardcoded into the linux kernel files to search for systemd via preset paths.

Core Concept 3.5: Package Manager

Packages are essentially a compressed archive of the follow three components.

The Binary: The no extension machine code files (such as the python3 executable)

Metadata: Information about the version of the package, the creator, the description, etc. (the file type differs across different distros but usually plain text or YAML/JSON variant)

Dependencies: a specific field within the metadata file that uses boolean logic to express which other packages the relevant package needs in order to work.

For different distros, the package extensions differ. For ubuntu it is .deb, for redhat/fedora it is .rpm, and for macOS it is just .pkg.

Each distro maintains a repository of packages. This is essentially a massive online server containing thousands of precompiled, vetted, and secure packages.

A package manager is the middle man that can access these repositories and install whatever packages you'd like into your local. The greatest strength of the package manager is dependency handling (making sure each dependency and their respective versions are installed in the correct order and manner, which, done otherwise, would break the package you are trying to use).

Here are some examples across distros (these are like what homebrew is for macOS).

ubuntu -> apt fedora/redhat -> dnf arch linux -> pacman

Core Concept 3.6: Display Server

The display server is the foundation of the machine's GUI. And it's job is to coordinate the communication between the kernel and apps, after which the GUI would adjust according to the action you've done with hardware (like the mouse or keyboard).

input (mouse click on button) -> hardware -> driver -> kernel -> display server -> app recognizes an action -> display server -> kernel -> driver -> hardware -> output (button takes on the "pressed" appearance)

The X11 display server has been the standard since the 1980s, however the modern successor that people are pivoting towards is Wayland (modern distros like ubuntu and fedora have already switched over to Wayland from X11).

Core Concept 3.7: Window Manager

The window manager is the software that gives the outputted pixels on the screen a sense of organization. In the linux world, window managers are separated into three categories

Stacking (floating) window managers: windows float on top of each other (this is how macOS and windows is by default)

Tiling window managers (TWM): windows tile within the screen (yabai/skhd or hammerspoon are extensions in macOS that make the GUI a tiling set up)

Dynamic window managers: this is a hybrid between stacking and tiling window managers

The window manager is also responsible for the design of windows (close x symbol at the top, border of window, etc.). Without it, all application graphical interfaces will be mushed towards the top left corner (you won't be able to grab it because the very concept of grabbing exists within the WM and not the display server) of the GUI made possible by the display server.

Core Concept 3.8: Wayland: The Modern Standard

Now, so far we have described the display server and window manager being to separate piece of software. However, Wayland or, more accurately, the Wayland compositor is both a display server and window manager, thus its processes are much faster than the old X11 standard. The following are different types of wayland compositors.

Mutter (used for the Gnome DE) - stacking KWin (used for the KDE Plasma DE) - hybrid Sway (standalone) - tiling Hyprland (standalone) - tiling/dynamic

each different "flavors" have their own pros and cons.

Core Concept 3.9: Desktop Environment

The DE is a bundled suite of separate applications that are designed to look, feel, and talk to each other cohesively. A DE is a general collection of the following programs.

The window manager/compositor: usually Wayland like we have discussed

The Shell/Panel: A standalone program that run on top of the display server, the bar the top of bottom of the desktop that holds the time, wifi, bluetooth, app icons, etc.

File Manager: the user space application that acts as the interface between you and the file system stored in the hardware. this is like finder for macOS

hardware(ssd) -> driver -> kernel -> file manager -> file system interface (screen)

Different DEs use different file managers:

dolphin (KDE Plasma) nautilus (Gnome) Thunar (XFCE)

Settings Manager: This is the GUI that allows you manipulate how the kernel, drivers, and compositor behave without having to handle complex file written in C.

Core Apps: These are the basic apps that are preset within the environment such as the terminal emulator (kitty), text editor (nvim), system monitor (htop), the package manager GUI (app store), etc.

Core Concept 3.10: Daemons

Daemons are the background processes that run constantly in a machine and is launched by the init system (usually systemd). Daemons exist for various processes and their process names are usually ended with a "d"

cupsd -> printing sshd -> ssh bluetoothd -> bluetooth journald -> journal (logging)

A user can access and talk with a daemon via a control tool.

To talk to systemd -> systemctl To talk to the network daemon -> networkctl To talk to bluetoothd -> bluetoothctl

3.3 Linux File System Hierarchy

Core Concept 3.11: Root Directory & Overview

The starting point of the linux file system is called the **root directory**, represented as a single forward slash: `/`. Every single file or directory within the system exists under this directory. The following is an overview of the first layer of directories within `/`.

```

/ (Root)
├── bin → usr/bin --- Core System Directories
├── sbin → usr/sbin
├── lib → usr/lib
├── boot
├── etc
├── home --- User Data and Configuration
├── root
├── usr
├── opt
├── proc --- Virtual and Variable Files
├── sys
├── dev
├── run
├── var --- Variable/Persistent Data
├── tmp
├── media --- Media and Mounts
├── mnt
└── srv

```

Core Concept 3.12: Core System Directories

Core system directories are the **foundational** directories of the Linux filesystem, housing the essential binaries, libraries, and configuration files that the operating system requires to boot and function. Generally speaking, these directories are managed by the operating system itself and are **not meant to be modified directly by the user**.

(a) `/bin`: The bin directory (which stand for "binaries") is a **symlink** to `/usr/bin`. It's primary function is to store foundational machine code executable files intended to be available to be executed by **all users**. These files can generally be categorized into three groups.

- **Filesystem & file tools** encompasses file system manipulation and navigation executables commonly used in the terminal: `ls`, `cp`, `mv`, `rm`, etc.
- **Shells** are the actual shell executable files one executes in order to switch between shell types: `sh`, `bash`, `zsh`.
- **Process & system tools** refer to executables meant for manipulating system processes via the terminal: `kill`, `ps`, `su`, `df`, etc.

`/usr/bin` is one of the directories listed in the `$PATH` environment variable.

(b) `/sbin`: The sbin directory (standing for system binaries) is a **symlink** to `/usr/sbin` and functions as the location for machine code executables intended for the **system administrator** or **root user**. These files can generally be categorized into the following groups.

- **Filesystem management & maintenance** tools handle the creation, repair and integrity of disk

partitions: `fsck`, `mkfs`, `fdisk`, `resize2fs`, etc.

- **System boot & initialization** tools encompass binaries that are responsible for turning on and off the machine into and out of a function desktop or server environment: `init` / `systemd`, `grub-install`, `reboot`, `shutdown`, `halt`, etc.
- **Network configuration & security** refers to tools to change network hardware states or configure deep-level routing and firewalls: `iptables` / `nftables`, `ip`, `route`, `ethtool`, etc.
- **Kernel and hardware management** binaries are programs that allow the user to talk to the linux kernel and in turn with the physical hardware: `modprobe`, `lsmod`, `hwclock`, `swapon`, etc.
- **Administrative user management** executables are for managing the different capabilities and attributes of the machine's users: `useradd`, `usermod`, `userdel`, `chage`, etc.

`/usr/sbin` is one of the directories listed in the `$PATH` environment variable and is only executable under `sudo`.

- (c) `/lib`: The lib directory (short for library) is a **symlink** to `/usr/lib` and contains essential **shared libraries** in machine code format and kernel modules that provide the fundamental instructions require by the binaries in `/bin` and `/sbin` to interact with the linux kernel and hardware during boot and execution.

Shared libraries are a shared hierarchy of `.so` files that reside in memory and provide modular, reusable logic (functions) to multiple independently running applications through dynamic linking at runtime.

- (d) `/boot`: The boot directory is the location of three foundational operating system files.
- `vmlinuz` is the actual linux kernel developed by Linus Torvalds and is short for "Virtual Memory Linux [compressed] (z)"
 - `initrd` / `initramfs` is the initial RAM disk needed to provide initial drivers to reach the harddrive during machine initialization.
 - `grub/` is a subdirectory containing the configurations for the GRUB bootloader, the program that runs (after BIOS/UEFI) when the computer is turned on before it loads in the linux kernel into RAM.

Note that this bootloading process occurs prior to `systemd`, which is the first software run once the system is actually "alive".

- (e) `/etc`: The etc directory (short for *editable text configurations*) is storage for text configurations files for the entire operating system across all users.

For instance, whereas `/.zshrc` would configure the local instance of the zsh shell, `/etc/.zshrc` would set the default for every person who logs into the machine.

Core Concept 3.13: User Data and Configuration

User data and configuration directories are where the **human-facing** side of Linux lives, housing personal workspaces, user-installed software, and the configuration files that tailor the system to individual users.

- (a) `/home`: The home directory is the primary personal workspace of the user(s). Each user will have their own subdirectory (with their username as the title) within `/home`, such as `/home/john` or `/home/sally`, and each user's subdirectory contents **will not be accessible** by other users.

This is the directory containing workspaces familiar to all computer users—such as `Documents/`, `Downloads/`, `Desktop/`, `Pictures/`, `Music/`, etc.—as well as the custom dotfiles for the relevant user,

such as `.bashrc`, `.config/`, etc.

- (b) `/root`: This is the workspace of the superuser and houses primarily `.bashrc` (or `.zshrc`)—the specific terminal settings that only apply when you are acting as the superuser via `sudo -i` or `sudo su`—and `.ssh`, which are security keys used for system level automated tasks.
- (c) `/usr`: The `usr` directory (*short for Unix System Resources*) holds the vast majority of static, read-only software and files. Like we have covered in Core Concept 3.12, the real directories for the `/bin`, `/sbin`, and `/lib` symlinks live here. Additionally, `/usr` houses the following important subdirectories.
- `local/` is a secondary, mirrored file system designed as a “safe zone” for the system administrator to install software manually **without interference from the system’s package manager** (`apt`, `pacman`, `dnf`). The packages residing here are available to all users of the machine but isn’t part of the standard distribution’s repository of packages.
At face value, the subdirectories within `local/` are an exact mirror of subdirectories of `/usr`: `/usr/local/bin`, `/usr/local/sbin`, `/usr/local/lib`, and `/usr/local/share`.
 - `share/` is the central storage location for **read-only, architecture-independent data** such as `man`—manual pages for different executable commands—and `doc`—general documentation, license agreements, and README files for every software package installed. Other static files and directories regarding fonts, timezones, and icons for the operating system are stored here too.
 - `include/` stores `.h` files (*C/C++ header files*) which provides the technical definitions required to compile software that links to system libraries.
 - `src/` contains the source code (*the human-readable C and Assembly code*) of the operating system’s components, most notably the Linux Kernel `vmlinux`.
- (d) `/opt`: The `opt` directory (*short for Optional*) is a reserved location for self-contained, third-party software **bundles**—software that keeps all its files (*binaries, libraries, icons, and configuration*) inside a single-dedicated subdirectory rather than scattering different files into different locations, such as executables in `/usr/local/bin` and libraries in `/usr/local/lib`. The latter being the primary installation method covered previously.

The **monolithic** software installed within `/opt` is usually built by firms and corporations such as Chrome or Zoom.

Core Concept 3.14: Virtual and Variable Files

Virtual and variable files directories are not stored on disk like conventional directories, but are instead **dynamically generated** by the kernel at runtime to expose the live state of processes, hardware, and system resources.

- (a) `/proc`: The `proc` directory (*short for Process Information*) contains files that provide a standardized way for the user and user-space programs to see **what the Kernel is doing** and what **processes and softwares** are running. This directory is classified as a **virtual filesystem**, meaning that the filesystem exists entirely in the computer’s RAM rather than physical storage, thus has no physical persistence like the `home` directory has. In general, `/proc` subdirectories are categorized into the following.
- The numbered folders (PIDs)** represents each process that is currently running on the system. Recall from Core Concept 3.4 that the init system `systemd` is PID 1.

- **Global files** are those that are vital for system administration such as `/proc/cpuinfo` for details about the processor, `/proc/meminfo` for real-time RAM usage, or `/proc/uptime` for how long the system has been running.
- (b) `/sys` : The sys directory (*short for sysfs*) is another virtual filesystem (like `proc`) that provides a standardized way for the user and user-space programs to see the state of **hardware** and the **drivers** that manages the machine.
- (c) `/dev` : The dev directory (*short for devices*) contains **device files**, the interface that allows programs to actually **read from and write to** the physical hardware itself.
- (d) `/run` : The run directory (*short for runtime*) is a **temporary filesystem (tmpfs)** that stores **runtime data**, which is information that different processes (*init systems, daemons, and services*) create and store into files that reside in `/run`.

Core Concept 3.15: Variable/Persistent Data

Variable and persistent data directories are where the system writes information that needs to **survive across reboots**, such as logs, application state, and cached data, as opposed to the ephemeral runtime data covered in Core Concept 3.14.

- (a) `/var` : The var directory (*short for Variable*) is the storage location for files that are expected to grow, change, and—most importantly—persist across reboots
 - `/var/log` is where the system writes its **log**, the chronological record of events, processes, or messages that occur within the system.
 - `/var/lib` store the “state” of applications and packages so that its services remain functional across reboots. *If you have built up a database in an application, the actual data would get stored in `/var/lib`, which safely stores that data even across reboots.*
 - `/var/cache` is a persistent storage area for data that was expensive to generate or download, kept locally so that the system doesn’t have to do all of the work again when asked to generate or download a second time.
- (b) `/tmp` : The tmp directory (*short for Temporary*) is designed for files that are needed only for a very short period—such as the duration of a $\text{\LaTeX}$ file compilation or a single boot session. In most modern linux distros, `/tmp` is a **temporary filesystem (tmpfs)**, meaning that it is wiped automatically during reboot and is RAM-backed.

Core Concept 3.16: Media and Mounts

Media and mount directories are where the Linux filesystem exposes **external** storage devices and network services, acting as the designated attachment points for hardware and data that exists outside of the machine itself.

- (a) `/media` : The media directory is the standard location where temporary, removeable devices—such as USB sticks, SD cards, and external hard drives—**automatically** appear in the file tree.
- (b) `/mnt` : The mnt directory (*short for Mount*) is the standard location where temporary, removable devices **manually** appear in the file tree via the shell command `mount`, usually used only in cases where the user requires more direct control of hardware connections relative to the automatic method mentioned previously in `/media`.

- (c) `/srv` : The `srv` directory (*short for Service*) is the designated home for data that the system **serves** to external users or other systems, used when the machine is acting as a host for other machines to connect to, such as files for a web server in `/srv/www`.

3.4 Partitioning & Disk Management

3.5 The Boot Process

3.6 Display Server & GPU Drivers

3.7 Networking Setup

3.8 Package Management

3.9 Major Linux Distributions

3.9.1 Ubuntu

placeholder text

Core Concept 3.17: Ubuntu Flavors
Kubuntu

3.9.2 Mint

placeholder text

3.9.3 Arch

placeholder text